15418 Final Project Report: Parallel Othello Solver

By Carolyn Liu and Jessica Qiu

Code: https://github.com/cliu585/mcts-othello
Website: https://cliu585.github.io/parallel-mcts-othello/

Summary

We developed a parallel Othello solver using the Monte Carlo Tree Search algorithm and analyzed its performance under parallelization strategies. The first approach used OpenMP multi-threading on the CPU for root parallelism to run independent searches from the same root state and merge the root results at the end. We also implemented a variant of this approach using virtual loss. The second approach uses leaf parallelism, where a shared MCTS tree is used and only simulations are done in parallel, after which rewards are returned and we backpropagate up the tree.

Background

The Monte Carlo Tree Search algorithm (MCTS) is a heuristic search algorithm designed to find optimal decisions in large state spaces, most notably for the purpose of playing games. It emerged in the mid-2000s as a breakthrough in game-playing AI: before MCTS, complex games such as Go were extremely difficult because traditional search couldn't handle the enormous branching factor. In 2016, it was combined with neural networks by Google Deepmind to create the computer programs AlphaGo, AlphaGo Zero, and AlphaZero, which were the first programs that could beat a professional human player without handicaps in a standard game.

The primary data structure is a game tree composed of nodes, where each node represents a game state. The core idea of MCTS is to build a search tree incrementally through repeated random simulations, using the outcomes to guide exploration toward more promising moves. Rather than exhaustively evaluating all possible game states, MCTS strategically samples the search space by balancing exploitation of known good moves with exploration of uncertain alternatives. The basic algorithm has four steps:

1. Selection: from the input, the current game state (the root), choose different moves to traverse down the game tree until you reach a leaf node, an unexplored state.
2. Expansion: in the unexplored state, if the game is not yet ended, add child nodes corresponding to possible moves from the game state in the leaf node. Choose one new child node *C*.
3. Simulation: from the new node *C*, complete a full simulation of the game; that is, play the game out until the very end and return the result. Moves can be chosen as desired.
4. Backpropagation: use the result of the played out game to update the information in the nodes from the root to *C*, incrementing visit counts and updating win statistics as necessary.

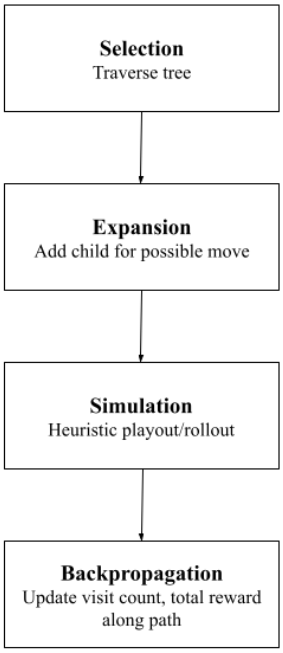


Figure 1: MCTS flow pipeline

Each iteration consists of one complete cycle through the four MCTS phases. Over many simulations, the tree grows asymmetrically: more frequently visited nodes having more children expanded, so that more effort is spent on promising playouts, instead of exhaustive or uniform search. The output is an enhanced search tree with accumulated statistics at each node, from which the best move is selected. One basic estimation of this is the child of the root with the highest visit count, as this represents the most thoroughly explored and statistically reliable option.

Parallelization

MCTS is inherently difficult to parallelize due to its sequential dependencies. Stages like selection, expansion, and backpropagation depend on up-to-date tree statistics, and the branches may expand at different rates. The game simulations also vary in length and branching can create irregular execution patterns and divergent control flow, which are challenging for SIMD-style hardware like GPUs. Also, tree accesses are often noncontiguous, limiting memory locality, and moving leaf states to the GPU can increase communication overhead and thus the communication to computation ratio as well. These factors make efficient parallel mapping difficult.

Overall, the workload exhibits hierarchical parallelism at multiple levels. Each step of the MCTS flow has different opportunities for parallelism as follows:

1. Selection: multiple independent root parallel searches can be run concurrently, each building a separate tree from the same root.

2. Expansion: this is generally lightweight, but can still be executed concurrently on multiple threads if leaf nodes are expanded in a batch.
3. Simulation: because each rollout is independent, multiple rollouts can be executed in parallel, either across leaf nodes or across multiple simulations of the same leaf.
4. Backpropagation: when running multiple independent trees for root parallelism, backpropagation can occur simultaneously for each tree on separate threads.

Notably, the simulation phase dominates computational cost, as each rollout involves playing an entire game from a leaf node to termination through random move selection. With games like Othello having moderate branching factors and game lengths, thousands or tens of thousands of simulations are needed to build reliable statistics.

Dependencies exist primarily in the tree traversal and update phases. Selection must read current node statistics to select which nodes to traverse, creating read dependencies on visit and win counts. Backpropagation writes to these same statistics, creating write-after-read hazards if multiple threads traverse overlapping paths. Expansion has potential write conflicts if multiple threads attempt to expand the same node simultaneously, requiring locks or atomic operations. Thus, The algorithm exhibits limited data parallelism because tree traversal is inherently serial—each node selection depends on the previous level's decision. However, simulation rollouts demonstrate excellent data parallelism since each game playout is independent with no shared state.

Locality characteristics are mixed. The tree traversal exhibits poor spatial locality because child node pointers can reference arbitrary memory locations, creating scattered access patterns unsuitable for cache optimization or SIMD vectorization. Similarly, simulations of different games are fundamentally not amenable to SIMD execution due to divergent control flow: different simulations take different paths through the game tree with varying numbers of moves before termination, and the heavy use of conditionals for move validation and game-end detection would cause significant SIMD lane divergence. However, within a single rollout there is temporal locality because the game state is copied and repeatedly modified, keeping data in cache. Even so, cross-simulation locality is poor since each rollout operates on independent state copies.

Approach

The core MCTS implementation uses C as the programming language with OpenMP for CPU parallelization. In particular, OpenMP was chosen for its ease of integration with existing C code and its efficient support for fork-join parallelism on multi-core CPUs. The hardware platform for our project is the Intel i7-9700 CPUs on the GHC cluster machines.

Notably, in the selection phase we chose to traverse the tree from root to leaf using the UCB1 algorithm, which computes a score that balances exploitation – prioritizing moves with high win rates – and exploration – investigating moves with high uncertainty due to low visit counts – for each child node.

From the baseline sequential implementation, we explore two main avenues of parallelization: a root-parallel approach and a leaf-parallel approach.

Root parallelism

Our basic root-parallel implementation maps independent MCTS search trees to separate CPU threads. At the start, we clone the root node to create identical tree copies for each OpenMP thread. Each thread then executes complete MCTS iterations (selection, expansion, simulation, backpropagation) entirely independently on its local tree copy using a thread-specific random seed to ensure different exploration paths. Notably, each thread also uses its own random seed, ensuring that simulations execute independently without any shared-memory conflicts.

Since each CPU thread owns its entire tree in private memory, synchronization overhead during the search is eliminated and threads do not need to communicate. After all threads complete their iterations, a sequential merge phase aggregates statistics from all thread-local trees back to the main root. This mapping trades memory efficiency, storing multiple tree copies, for parallelism efficiency – there is no lock contention or atomic operations during the search phase.

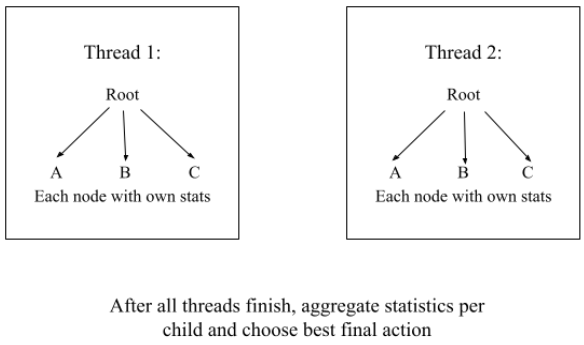


Figure 2: Root parallelism outline

Virtual loss

We attempt to improve upon this basic root parallelism by having all threads share a single tree structure, avoiding memory and search duplication. In particular, we use virtual loss penalties to prevent multiple threads from simulating the same node at the same time, ideally reducing contention and unnecessary work while also making exploration more diverse. Each MCTS iteration is assigned to a thread via using dynamic OpenMP scheduling, to handle the irregular computation patterns arising from variable game lengths, and threads traverse the shared tree concurrently during selection. As a thread descends through each node, it atomically increments the visit count and atomically decrements the wins by the virtual loss penalty value. This temporary pessimistic adjustment discourages other threads from immediately following the same path, promoting exploration diversity.

When a thread reaches a leaf requiring expansion, it acquires an OpenMP lock to exclusively expand that node, preventing race conditions where multiple threads might try to create children

simultaneously. After expansion, the lock is released, and the thread continues to select one of the newly created children. After completing the simulation, the thread performs backpropagation by walking back through its recorded path, using atomic operations to update each node to both remove the virtual loss penalty and add the actual simulation result.

This mapping is more complex but memory-efficient—all threads share the same tree in memory, with per-node locks coordinating expansion and atomic operations handling concurrent reads and writes to visit counts and win statistics. The UCB1 calculation is modified to use atomic reads of node statistics to get consistent snapshots despite concurrent updates by other threads.

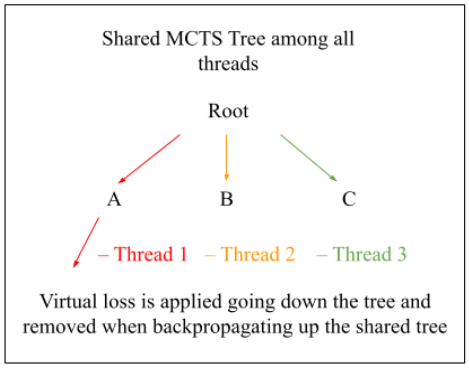


Figure 3: Root parallelism with virtual loss outline

Leaf parallelism

We also implemented leaf parallelism also on the CPU with multithreading. In this approach, the selection and expansion steps of MCTS remain serial. The algorithm begins by dividing the total number of desired simulations into groups, where each group performs a fixed number of parallel rollouts. For each group, the main thread performs the serial selection phase and once at a leaf, the algorithm performs serial expansion: if the node has been previously visited and there are still available legal moves, it generates children, and then randomly selects one of them as the expansion result. This single expanded node will be where all parallel rollouts will begin.

The rollout phase is parallelized using OpenMP. Each thread receives its own private copy of the leaf board state and uses its own random seed, ensuring that simulations execute independently without any shared-memory conflicts. Inside the parallel region, every thread performs a full playout and then performs backpropagation on the shared tree, ascending from the selected leaf node back to the root and updating each ancestor's statistics. To avoid data races during backpropagation, we used OpenMP's atomic directive for both the visit counter and accumulated win score.

This structure required modifying the traditional serial MCTS control flow. Instead of performing one selection, one simulation, and one backpropagation per iteration, our implementation performs a single selection and expansion per group, followed by a burst of parallel simulations. This batching design maps naturally to multicore CPUs as the expensive part, simulation, is distributed across all hardware threads, while the inexpensive but

synchronization-sensitive components stay serial. Across the entire computation, only two parts of the tree require coordination: the selection/expansion path (handled serially) and the atomic backpropagation updates. By ensuring backpropagation uses only atomic operations instead of mutex locks, we kept synchronization overhead low. Overall, the algorithm effectively leverages CPU parallelism by keeping the shared tree small, isolating expensive computation inside thread-local rollouts, and performing only lightweight atomic operations when writing results back to the tree.

We took a few tries to get to our current version of leaf parallelism, as originally we thought this strategy would be fit for the GPU environment using CUDA. In our original idea, we thought that the CPU could just handle the shared tree for selection, expansion, and backpropagation, while the independent game simulations (rollouts) from newly expanded leaf nodes could be performed by the GPU in parallel. Since the workload of rollouts is massively parallel, each rollout follows the same algorithm and threads can execute the same instructions so it's uniform and independent. Thus, there would be minimal synchronization and predictable memory access, which would be the type of computation the GPU is designed to accelerate.

However, we couldn't achieve this goal since through experimenting we found that speedups were small. Thus, we migrated our leaf-parallel implementation to the CPU with OpenMP.

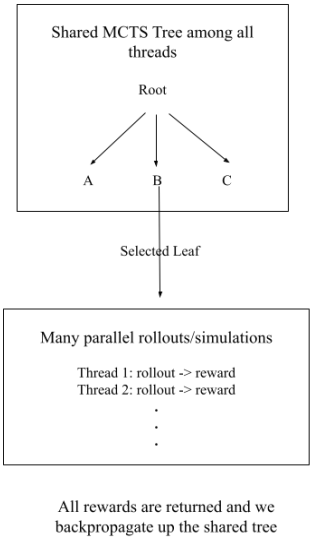


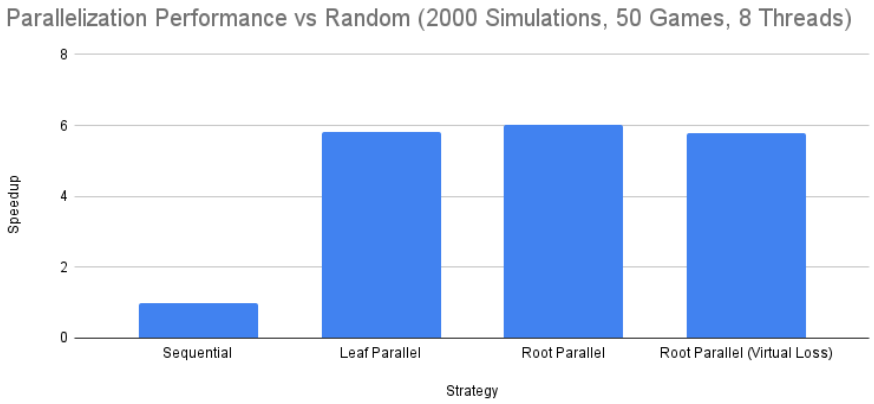
Figure 4: Leaf parallelism outline

Results

Our performances were measured by the total time speedups compared to the sequential baseline of one thread. We also measured average time per move and win rate for correctness. For benchmarking, we tested different simulation sizes, and number of threads and our inputs were game states on a Othello board. We also measured the average time spent on different phases of the algorithm. We also tested the efficacy of the different MCTS algorithms against each other. All results were obtained from the GHC machines.

Parallelization Performance vs Random (2000 Simulations, 50 Games, 8 Threads)

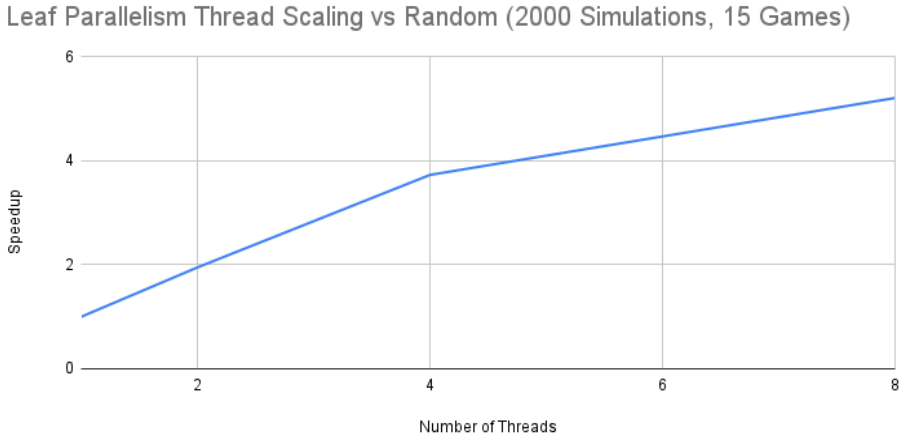
Strategy	Win Rate	Average Time/Move (s)	Speedup
Sequential	100.0%	0.0526	1.00x
Leaf Parallel	100.0%	0.009	5.82x
Root Parallel	100.0%	0.0087	6.02x
Root Parallel (Virtual Loss)	100.0%	0.0091	5.78x



Thread Scaling vs Random (2000 Simulations, 15 Games)

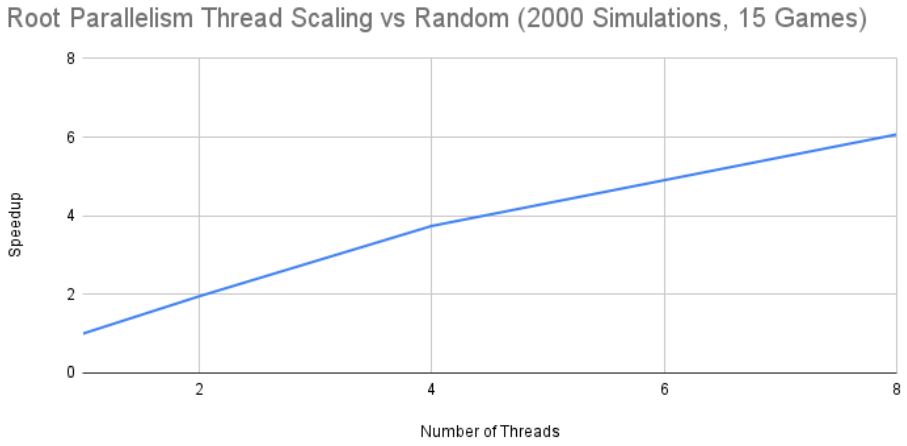
Leaf Parallel:

Number of Threads	Average Time/Move (s)	Speedup
1	0.0526	1.00x
2	0.0269	1.95x
4	0.0141	3.73x
8	0.0101	5.21x



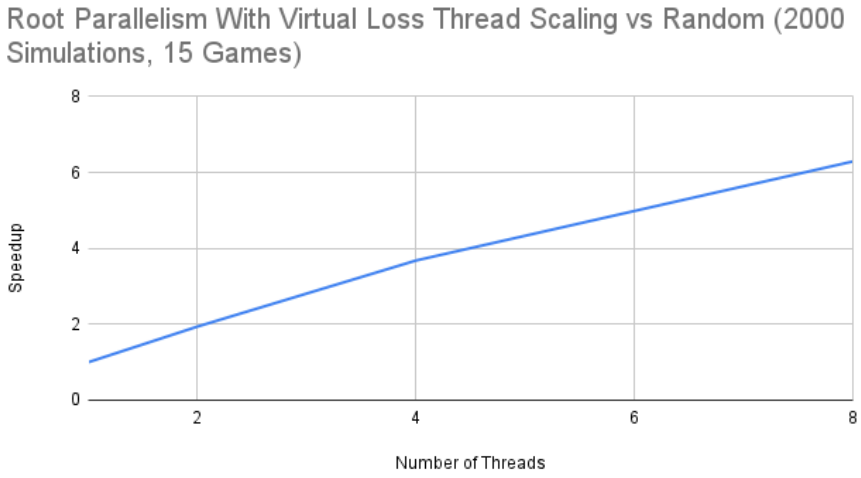
Root Parallel:

Number of Threads	Average Time/Move (s)	Speedup
1	0.0523	1.00x
2	0.0269	1.95x
4	0.014	3.74x
8	0.0086	6.07x



Root Parallel (Virtual Loss):

Number of Threads	Average Time/Move (s)	Speedup
1	0.0547	1.00x
2	0.0281	1.94x
4	0.0149	3.68x
8	0.0087	6.29x



Head to Head (30 Games, 8 Threads):

A vs B	**A Win %**	**B Win %**
Sequential vs Leaf Parallel	60%	36.7%
Sequential vs Root Parallel	53.3%	43.3%
Sequential vs Root Parallel (Virtual Loss)	50%	46.7%
Leaf Parallel vs Root Parallel	46.7%	53.3%
Leaf Parallel vs Root Parallel (Virtual Loss)	33.3%	63.3%
Root Parallel vs Root Parallel (Virtual Loss)	43.3%	53.3%

Simulation Scaling Win Count vs Random (30 Games, 8 Threads):

Number of Simulations	**Sequential**	**Leaf Parallel**	**Root Parallel**	**Root Parallel (Virtual Loss)**
500	30/30	30/30	30/30	30/30
1000	30/30	30/30	30/30	30/30
2000	30/30	30/30	30/30	30/30
5000	30/30	30/30	30/30	30/30

Analysis

Leaf parallelism

Leaf parallelism had the second-best performance over the sequential implementation, but showed the weakest thread scaling among the three approaches. This was likely a result of several compounding factors.

One factor may have been insufficient parallelism granularity: the limited number of rollouts may have resulted in insufficient work per parallel region, ultimately failing to amortize OpenMP's thread synchronization costs. In particular, the frequent entering and exiting of parallel regions – once per iteration group – adds overhead that becomes more significant as thread count increases. Thus, the added overhead results in the degraded thread scaling for leaf

parallelism, despite achieving similar performances at low threads when compared to root parallelism.

Additionally, between parallel rollout batches, the algorithm performs selection and expansion to find the next leaf. These phases are inherently serial—each node selection depends on the previous level's decision, and expansion requires checking all legal moves and creating node structures. As the tree grows deeper and wider through the course of a game, these sequential phases consume increasing time. Thus, this sequential execution limits the total speedup that the algorithm can achieve by Amdahl's law.

Moreover, all rollouts from the same leaf must backpropagate results up the tree simultaneously, creating hotspot contention at nodes near the root where all paths converge. Since each update to the node statistics is atomic, this especially increases cache coherence traffic and further adds to communication costs. Game simulations also vary in length, so this creates load imbalance between the various threads, although this is somewhat mitigated by our usage of OpenMP's dynamic scheduling.

Notably, leaf parallelism had the worst win rates. This is likely because the state at each node was stale: by batching all rollouts from a single leaf before backpropagating, the algorithm delays incorporating simulation results into the tree statistics. In contrast, sequential MCTS immediately backpropagates after each simulation, allowing subsequent iterations to leverage updated statistics for better move selection. This delay causes the algorithm to make expansion decisions based on inaccurate information, resulting in a degraded win rate.

Root parallelism

The basic root parallelism implementation achieved the best speedup over the sequential implementation. This is likely because each thread executes completely independently on its own tree copy with zero synchronization during search, eliminating the atomic operation overhead and lock contention that plague other approaches. However, the speedup was still sublinear, likely because of some of the limitations of this design.

In basic root parallelism, after all threads complete their independent searches, a single thread must sequentially merge statistics from all thread-local trees back to the main root. By Amdahl's law, this fundamentally limits the maximum speedup that this algorithm can achieve.

Moreover, because each thread maintains its own complete tree copy, the memory footprint is many times larger than the sequential version. This not only added to the initial overhead of allocating this memory, but also potentially saturated the memory bandwidth as multiple threads competed for memory access. This is especially notable because the locality characteristics are mixed, so caching has limited benefits. In particular, tree traversal exhibits poor spatial locality because child node pointers can reference arbitrary memory locations, creating scattered access patterns unsuitable for cache optimization. Thus, the memory bandwidth created a bottleneck, decreasing the speedup that this algorithm could achieve.

Additionally, because each thread's trees are independent, there is redundant exploration when multiple threads investigate the same promising paths. This problem is exacerbated by the fact that we are using UCB1, which favors high win-rate moves. As a result, there is wasted computational effort being spent by each thread exploring similar paths.

Root parallelism with virtual loss

Root parallelism with virtual loss achieved the weakest speedup over the sequential implementation. The virtual loss penalties changed the search patterns of the algorithm, potentially disturbing the balance between exploitation and exploration of the UCB1 move selection. Also, the increased search diversity reduced redundant exploration compared to basic root parallelism. It also means more nodes get expanded across the tree. As a result, there is more memory allocation, more cache pressure, and potentially more time spent in less-promising regions of the tree. However, root parallelism with virtual loss outperformed both leaf parallelism and basic root parallelism in win rates across all simulation counts, so the increased search diversity may have contributed to the better decision quality despite the slower computation.

However, the thread scaling was one of the best. While virtual loss does introduce additional synchronization overhead, this approach more effectively distributes work across threads by reducing contention for the same tree paths. Notably, each atomic operation still triggers cache coherence protocol traffic across CPU cores, which reduces effective memory bandwidth and increases memory latency. However, the virtual loss mechanism's ability to guide threads toward different regions of the search tree outweighs this overhead as thread count increases, leading to better scaling characteristics.

Deeper analysis

We were able to measure the number of wins across different simulation sizes, but didn't see any pattern of increase in the number of wins. This suggests that increasing the number of rollouts alone does not affect the correctness of the algorithm's decisions. In other words, the MCTS implementation produces correct moves regardless of simulation size, and factors such as move strategy likely play a larger role in determining outcomes.

Notably, when pitting the different MCTS algorithms against each other, the sequential MCTS algorithm outperformed the others, followed by root parallelism with virtual loss, then root parallelism, then leaf parallelism. This is likely because MCTS immediately backpropagates simulation results after each rollout. This therefore allows UCB1 to make optimal selection decisions without the staleness, redundancy, or artificial penalties that parallel approaches introduce. On the other hand, the search diversity encouraged by the virtual loss penalties prevent the algorithm from getting stuck in any local maxima of best moves, and leaf parallelism suffers from stale statistics due to its batched updates.

Strategy	Selection	Expansion	Simulation	Backpropagation
Sequential	1.15%	3.64%	95.13%	0.08%
Leaf Parallel	0.04%	0.17%	98.92%	0.87%
Root Parallel	0.74%	3.91%	95.27%	0.08%
Root Parallel (Virtual Loss)	1.86%	11.17%	86.56%	0.41%

Figure 5: MCTS phase execution breakdown for 8 threads

We also divided our algorithm into the 4 phases of MCTS and measured the execution time for each. We can see from the table above that the most computationally intensive component for all the strategies was by far the simulation phase. This makes intuitive sense: each simulation runs a complete game to termination, being inherently computation-heavy due to the repeated game logic execution and operations.

Notably, the higher backpropagation time for leaf parallelism reflects the contention due to atomic operations, where all rollout threads must simultaneously update the same path back to the root node, creating cache coherence traffic and lock contention. The increased time spent in the simulation phase indicates that the parallelism overhead could be improved.

On the other hand, root parallelism has an extremely similar percentage of time spent in each phase compared to the sequential implementation. This is consistent with the algorithm's design: each thread essentially runs sequential MCTS independently on its own complete tree copy, so the overall proportion of time spent is the same.

In contrast, root parallelism with virtual loss spends significantly more time in the expansion phase compared to all of the other strategies. This is likely due to the lock contention in the expansion phase creating synchronization overhead. Moreover, the increased search diversity also means that the algorithm encounters and expands significantly more unique nodes throughout the search tree. Each new expansion requires memory allocation for node structures and children arrays, increases cache pressure from scattered memory access patterns, and adds computational overhead from the expansion operation itself.

CPU vs. GPU leaf parallelism

Notably, the results were significantly better for the CPU implementation of leaf parallelism rather than the GPU version. Our hypothesis is that MCTS expands the tree progressively, so early in the search there are only a few leaves, and each leaf receives only a small number of rollouts. This results in very small GPU kernels that fail to fully utilize the hardware. Our simulation logic is also relatively complex – with a long loop – and branch-heavy, causing each GPU thread to do mostly sequential work and introducing warp divergence, which could further reduce parallel efficiency. We attempted to simplify the code by introducing a max depth limit to

reduce the amount of sequential work, but while this improved our speedups a little bit, it was still low compared to our CPU versions and furthermore, our wins were less. On the CPU, OpenMP allows multiple threads to efficiently handle the branched, irregular simulation logic with minimal overhead, providing better parallel performance and more consistent win rates. Thus, we believe our choice of machine target was sound.

References

- MCTS algorithm tutorial: https://github.com/ai-boson/mcts
- Reversi Monte Carlo Tree Search: https://gitlab.com/royhung_/reversi-monte-carlo-tree-search
- Reversi AI experiments: https://github.com/psaikko/mcts-reversi
- MCTS GPU parallelization in Python: https://www.sciencedirect.com/science/article/pii/S2352711025001062
- Parallelizing MCTS: https://www-users.cse.umn.edu/~gini/publications/papers/Steinmetz2020TG.pdf
- Virtual loss in MCTS: https://liacs.leidenuniv.nl/~plaata1/papers/paper_ICAART17.pdf

Distribution of Work

Total credit: Carolyn - 50%, Jessica - 50%

Carolyn:

- Implemented basic Othello game logic and sequential MCTS algorithm with UCB
- Designed and implemented MCTS root parallelism with OpenMP
- Designed and implemented MCTS root parallelism with virtual loss, OpenMP
- Worked on the benchmarking scripts for calculating performance
- Ran experiments on root parallelism parameters and analyzed the final results

Jessica:

- Implemented basic Othello game logic and sequential MCTS algorithm with UCB
- Designed and attempted implementing GPU version of MCTS leaf parallelism with CUDA
- Designed and implemented CPU version of MCTS leaf parallelism with OpenMP
- Worked on the benchmarking scripts for calculating performance
- Ran experiments on leaf parallelism parameters and analyzed the final results